

Uploading Findings to ArmorCode

Four methods via the ac-sdk-v2 Python SDK

There are four distinct ways to upload findings into ArmorCode from Python. They differ by input format (JSON objects, CSV, or a native scanner report) and by transport (a direct POST vs. a presigned-S3 upload). Only Method 1 is wrapped in the SDK today; the others are called via `ac_session` until wrapped.

Methods at a Glance

#	Method	Endpoint	Input	In SDK?
1	Generic JSON	POST /api/findings/upload	JSON array of findings	Yes — <code>upload_findings()</code>
2	CSV multipart	POST /user/findings/upload/csv	CSV file (multipart)	No
3	CSV → custom tool	POST /user/tools/generic/configurations/{tool_name}/upload	CSV mapped to a tool config (presigned S3)	No
4	Native scan report	POST /api/v2/scans/upload/initiate → presign → complete	Raw scanner output (multipart S3)	No

How to Choose

- **Method 1 (Generic JSON):** Finding objects built in code.
- **Method 2 (CSV multipart):** A normalized findings CSV uploaded in one shot.
- **Method 3 (CSV → custom tool):** A CSV attributed to a specific custom tool with a saved field mapping, and/or large files.
- **Method 4 (Native scan report):** A raw native scanner report that ArmorCode should parse itself.

Method 1 — Generic JSON (implemented)

POST a JSON array of findings in ArmorCode's Generic Finding format. `product`, `sub_product` and `environment` are query params; names resolve to ids automatically. The sub-product must belong to the product or the API returns a 500.

```
ac.upload_findings(  
    {"Title": "Hardcoded secret", "Severity": "High",  
     "Description": "...", "ToolFindingId": "unique-001",  
     "Category": "SECURITY"},
```



```
product="my-product", sub_product="my-subproduct",  
environment="Production",  
)  
# -> {"scanId": 137746107} # ingest is async
```

Method 2 — CSV Multipart (not wrapped)

Direct multipart/form-data upload of a CSV file — no S3 hop. Simplest CSV path when you don't need custom-tool attribution.

```
files = {"file": ("findings.csv", open("findings.csv", "rb"), "text/csv")}  
resp = ac._session.post(  
    f"{ac.base_url}/user/findings/upload/csv", files=files)  
resp.raise_for_status()
```

Method 3 — CSV → Custom Tool via Presigned S3 (not wrapped)

Uploads a CSV bound to a named custom tool configuration (with its saved field mapping). Body is a ScanUploadRequest requiring product (int id), subProduct (int id), environment, and fileName. The call returns a signedUrl; PUT the raw CSV bytes to it with no auth header (the signature is in the URL). Columns are always comma-separated; the field delimiter (; or |) only splits multi-value cells.

```
body = {"product": 843798, "subProduct": 530718,  
        "environment": "Production", "fileName": "sca.csv",  
        "customTool": True}  
tool = "custom-SCA-Sample-Findings"  
r = ac._session.post(  
    f"{ac.base_url}/user/tools/generic/configurations/{tool}/upload",  
    json=body)  
url = r.json()["signedUrl"]  
requests.put(url, data=open("sca.csv", "rb").read(),  
             headers={"Content-Type": "text/csv"})
```

Alternative endpoint: POST /api/v2/findings/csv/upload uses MultiScanUploadRequest (required fileName, toolName).

Method 4 — Native Scan Report (not wrapped)

For a raw scanner output file that ArmorCode parses itself. Three-step multipart-to-S3: initiate (query params toolName + totalParts) → presign each part → complete with the returned ETags. Legacy single-shot variant: POST /api/scanUploadUrl.

```
init = ac._session.post(  
    f"{ac.base_url}/api/v2/scans/upload/initiate",  
    params={"toolName": "Snyk", "totalParts": 1},  
    json={"product": 843798, "subProduct": 530718,  
        "environment": "Production", "fileName": "snyk.json"})  
.json()  
# then: presign each part -> PUT to S3 -> complete
```

Verification Status

All four methods were driven end-to-end against JulianSandbox (July 2026) and succeeded. Runnable examples for all four live in examples/upload_findings.py in the SDK repo.

Method	Sandbox result
1. Generic JSON	Success — scanId returned
2. CSV multipart	HTTP 200 — 'File uploaded successfully'
3. CSV → custom tool (custom-SCA-Sample-Findings)	presign 200 + S3 PUT 200
4. Native scan report (initiate)	HTTP 200 + uploadId

Notes

- **Method 2 (CSV multipart):** Do NOT send Content-Type: application/json. The SDK's session pins that header, which breaks the multipart boundary and returns HTTP 415. Send only the auth header and let the HTTP library set the multipart content-type.
- **Method 4 (native scan report):** toolName must be a NATIVE ArmorCode scanner (Snyk, Trivy, Semgrep, SonarQube, Dependabot all tested). A custom tool name will be rejected with 400 'Custom tool is not supported yet'.